

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Основные возможности библиотеки Playwring в Python

Выполнил обучающийся:

Груздев Николай александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Получить практические навыки по работе с Playwring.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Настройка окружения и создание теста.

Для начала создадим виртуальное окружение Python, установим в него библиотеку Playwright и необходимые браузеры (рис. 1-2).

```
user@vm1:~/playwright$ source venv/bin/activate
(venv) user@vm1:~/playwright$ pip install playwright
Collecting playwright
  Downloading playwright-1.57.0-py3-none-manylinux1_x86_64.whl (46.0 MB)
    46.0/46.0 MB 1.8 MB/s eta 0:00:00
```

Рисунок 1 – Установка Playwright

```
(venv) user@vm1:~/playwright$ python -m playwright install
Downloading Chromium 143.0.7499.4 (playwright build v1200) fr
```

Рисунок 2 – Установка браузеров

Далее напишем скрипт, который проверяет сценарии успешной и неудачной авторизации на веб-странице Apache2 (листинг 1).

Листинг 1 – Файл test.py

```
from playwright.sync_api import sync_playwright
import time
with sync_playwright() as playwright:
    browser = playwright.firefox.launch(channel="firefox",
    headless=False)
    context = browser.new_context(ignore_https_errors=True)
    page = context.new_page()
    print("Выполнение авторизации с неверными данными...")
    page.goto('http://vm1:82')
    page.fill('input[name="login"]', 'aaa')
    page.fill('input[name="password"]', 'bbb')
    page.click('button[type="button"]')
    try:
        login_info = page.get_by_text('Wrong login!')
        print(f"Успешно получена ошибка {login_info.text_content()}")
    except:
        print("Ошибка авторизации не сработала")
        time.sleep(2)
    print("Выполнение авторизации с правильными данными...")
    page.goto('http://vm1:82')
    page.fill('input[name="login"]', 'user')
    page.fill('input[name="password"]', '1111')
    page.click('button[type="button"]')
    try:
        new_page_info = page.get_by_text('Apache2 Debian Default
Page')
```

```
        print(f"Успешно выполнен вход на страницу  
{new_page_info.text_content()}")  
    except:  
        print("Авторизация не сработала")  
        time.sleep(2)  
        page.close()  
        browser.close()
```

Выполнение теста.

После запуска все проверки отработали успешно (рис. 3). В консоли были выведены соответствующие сообщения о результатах тестирования (рис. 4).

Login page

Wrong login!

Enter password

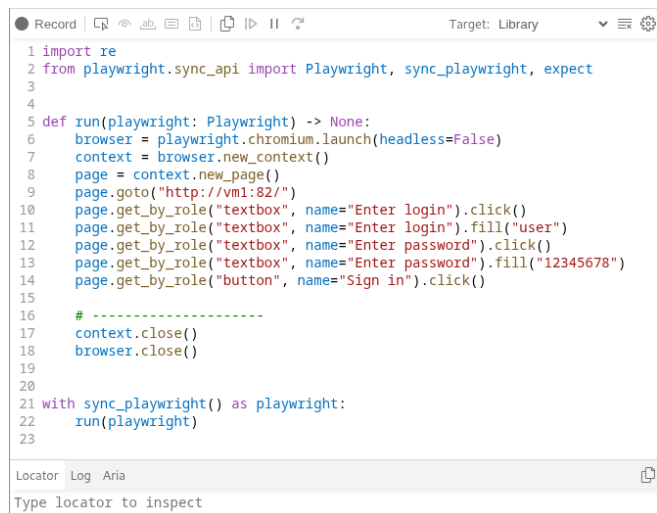
Рисунок 3 – Процесс выполнения теста

```
(venv) user@vm1:~/playwright$ python test.py  
Выполнение авторизации с неверными данными...  
Успешно получена ошибка Wrong login!  
Выполнение авторизации с правильными данными...  
Успешно выполнен вход на страницу  
Apache2 Debian Default Page
```

Рисунок 4 – Результаты теста

Генерация кода.

Для автоматизации создания скрипта была использована команда `playwright codegen http://vm1:82/`. После выполнения действий ввода логина, пароля и нажатия кнопки в интерактивном режиме Playwright сгенерировал готовый код на Python (рис. 5).



The screenshot shows the Playwright Codegen interface. At the top, there's a toolbar with icons for recording, pausing, and other actions. Below the toolbar, a code editor displays a Python script generated by the tool. The script imports the necessary Playwright modules and defines a function to perform a login action. The script includes comments indicating the steps recorded during the session. At the bottom, there's a 'Locator' section with a dropdown menu set to 'Log' and a text input field containing 'Aria'. A placeholder text 'Type locator to inspect' is visible below the input field.

```
1 import re
2 from playwright.sync_api import Playwright, sync_playwright, expect
3
4
5 def run(playwright: Playwright) -> None:
6     browser = playwright.chromium.launch(headless=False)
7     context = browser.new_context()
8     page = context.new_page()
9     page.goto("http://vm1:82/")
10    page.get_by_role("textbox", name="Enter login").click()
11    page.get_by_role("textbox", name="Enter login").fill("user")
12    page.get_by_role("textbox", name="Enter password").click()
13    page.get_by_role("textbox", name="Enter password").fill("12345678")
14    page.get_by_role("button", name="Sign in").click()
15
16    # -----
17    context.close()
18    browser.close()
19
20
21 with sync_playwright() as playwright:
22     run(playwright)
23
```

Locator Log Aria

Type locator to inspect

Рисунок 5 – Генерация кода через Playwright Codegen

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Самодокументирование — это мощный инструмент для тестировщиков и разработчиков, позволяющий быстро создавать автоматизированные тесты без необходимости ручного написания кода;

2. Selenium, в отличие от Playwright, имеет более сложный API, что может затруднять обучение для начинающих. Также он может работать медленнее при выполнении тестов и иметь проблемы совместимости с некоторыми версиями браузеров;

3. Postman специализируется на тестировании API и работе с HTTP-запросами, предлагая удобный веб-интерфейс и требующий регистрацию. Playwright же ориентирован на тестирование веб-интерфейсов и взаимодействия с элементами страницы.

ВЫВОД

В ходе работы были получены практические навыки по настройке и использованию фреймворка Playwright для автоматизации тестирования веб-приложений.